

Albertian Institute of Science & Technology

(Approved by AICTE, Affiliated to State Board of Technical Education)

Lab Manual for

Python Programming Lab – 3347

Semester – III (AI & ML)

Prepared By,

Kalyani M Biju

Lecturer, Department of Artificial Intelligence & Machine Learning

INDEX

Exp No.	Name of Experiment	Date	Page No.	Faculty Signature
1	Area Of Triangle			
2	Largest Among Three Numbers			
3	Palindrome Numbers			
4	Palindrome String			
5	Fibonacci Series			
6	Vowels And Consonants			
7	Sorting			
8	Maximum And Minimum Element Of A Tuple			
9	List Of Frequencies			
10	Frequency Of Characters Of A File			
11	Exception Handling			
12	Student Class			
13	Inheritance			
14	Numpy			
15	Panda Module			
16	Matplotlib			

EXPERIMENT NO: 1 **Date:** _____

AREA OF TRIANGLE

AIM

Write a program to Find the area of a right angled triangle

OBJECTIVES

- To understand fundamental user input operations and type casting in Python.
- To execute arithmetic multiplication and division calculations for geometry.
- To format and display computed numerical results to the console output.

ALGORITHM

Step 1 :- Initialize the execution environment to accept input values for base and height of the right angled triangle.

Step 2 :- Prompt the user to enter the base length of the triangle and convert it to an integer.

Step 3 :- Prompt the user to enter the height of the triangle and convert it to an integer.

Step 4 :- Multiply the base and height values together and store the result in variable c.

Step 5 :- Divide the product c by two to calculate the area of the right angled triangle.

Step 6 :- Output the calculated area value along with descriptive text to the standard console display window.

Step 7 :- Terminate the execution of the Python program successfully and release system memory resources.

PROGRAM

```
b = int(input("Enter the base"))
h = int(input("Enter the height"))
c = b * h
area = c / 2
print("Area of right angle triangle is:", area, "sqcm")
```

SAMPLE OUTPUT

Terminal Output

```
Enter the base:  5
Enter the height:  10
Area of right angle triangle is:  25.0 sqcm
```

RESULT

The program to calculate the area of a right-angled triangle was successfully implemented and verified.

EXPERIMENT NO: 2 **Date:** _____

LARGEST AMONG THREE NUMBERS

AIM

Write a program to Find the greatest among three numbers

OBJECTIVES

- To implement multi-way branching using conditional control structures.
- To utilize relational and logical AND operators for comparison.
- To determine maximum numerical value among user inputs.

ALGORITHM

Step 1 :- Begin execution of the program to compare three user input numbers and find the largest value.

Step 2 :- Prompt the user to enter the first number and convert the input string to an integer.

Step 3 :- Prompt the user to enter the second number and convert the input string to an integer.

Step 4 :- Prompt the user to enter the third number and convert the input string to an integer.

Step 5 :- Check if the first number is strictly greater than both second and third input numbers.

Step 6 :- Check if the second number is strictly greater than the third number if first check fails.

Step 7 :- Consider the third number as largest if both previous conditional checks evaluate to false.

Step 8 :- Print the identified largest number to the terminal display with an appropriate descriptive text message.

Step 9 :- Conclude the execution of the number comparison program and return control to the operating system.

PROGRAM

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a > b and a > c:
    print("largest number is ", a)
elif b > c:
    print("the largest number is ", b)
else:
    print("the largest number is ", c)
```

SAMPLE OUTPUT

Terminal Output

```
Enter first number:  10
Enter second number:  5
Enter third number:  2
largest number is 10
```

RESULT

The program to find the largest among three numbers was successfully executed and verified.

EXPERIMENT NO: 3 **Date:** _____

PALINDROME NUMBERS

AIM

Write a program to check whether a given number is palindrome or not

OBJECTIVES

- To implement indefinite iteration using while loop structure.
- To perform digit extraction using modulus and integer floor division.
- To reverse integer values and verify numeric symmetry.

ALGORITHM

Step 1 :- Begin the program to check whether a given user input integer number is a palindrome or not.

Step 2 :- Prompt the user to enter a number and store it in variable n and temporary variable temp.

Step 3 :- Set the initial value of the reversed number variable rev to zero before starting loop.

Step 4 :- Execute a while loop as long as the temporary variable temp is strictly greater than zero.

Step 5 :- Calculate the remainder digit by taking modulus of temporary variable temp by base value ten.

Step 6 :- Multiply rev by ten and add extracted remainder digit to accumulate reversed numerical value.

Step 7 :- Perform integer floor division on variable temp by ten to remove the last processed digit.

Step 8 :- Compare the reversed number rev with original number n for exact numerical equality match.

Step 9 :- Print palindrome if numbers are equal, otherwise print not palindrome to console display window.

Step 10 :- Terminate the execution of the palindrome check Python script and clear runtime variables from memory.

PROGRAM

```
n = int(input("enter a number"))
rev = 0
temp = n
while temp > 0:
    rem = temp % 10
    rev = rev * 10 + rem
    temp = temp // 10

if rev == n:
    print("palindrome")
else:
    print("not palindrome")
```

SAMPLE OUTPUT

Terminal Output

```
enter a number:  121
palindrome
```

RESULT

The palindrome number checking program was executed and verified for various input cases.

EXPERIMENT NO: 4 **Date:** _____

PALINDROME STRING

AIM

Write a program to check whether a given string is palindrome or not

OBJECTIVES

- To understand string indexing and slicing operations in Python.
- To reverse a text string starting from the last character to the first character.
- To compare the original string with the reversed string for palindrome validation.

ALGORITHM

Step 1 :- Initialize the execution environment to accept string input and evaluate string palindrome symmetry.

Step 2 :- Prompt the user to enter a text string and store the input in variable s.

Step 3 :- Reverse the string by slicing from back to front and store in variable rev.

Step 4 :- Compare the original string s with the reversed string rev for exact string equality.

Step 5 :- Display palindrome message if strings match, otherwise print not a palindrome message to console.

Step 6 :- Terminate the execution of the Python program successfully and free runtime resources back to system.

PROGRAM

```
s = input("Enter a string: ")
rev = s[::-1]
if s == rev:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

SAMPLE OUTPUT

Terminal Output

```
Enter a string:  madam
The string is a palindrome
```

RESULT

The program to check whether a given string is a palindrome was successfully implemented and output verified.

EXPERIMENT NO: 5 **Date:** _____

FIBONACCI SERIES

AIM

Write a program to print Fibonacci series

OBJECTIVES

- To understand sequence generation and iterative loop bounds.
- To manage variable swapping and accumulator updates during iterations.
- To display sequence terms up to user specified limit.

ALGORITHM

Step 1 :- Initialize the execution environment to compute and display a specified number of Fibonacci series terms.

Step 2 :- Prompt user to enter total number of terms and store integer value in variable n.

Step 3 :- Set first term variable a to zero and second term variable b to value one.

Step 4 :- Print the first two seed numbers a and b on separate lines to console.

Step 5 :- Loop through index range starting from two up to specified total term count n.

Step 6 :- Add current values of variables a and b to compute the next term c.

Step 7 :- Print the newly calculated Fibonacci term c to standard terminal console display screen.

Step 8 :- Update variable a with value of b, and update variable b with new value c.

Step 9 :- Terminate the execution of Fibonacci series generation program and free allocated system memory resources.

PROGRAM

```
n = int(input("enter your number of terms"))
a = 0
b = 1
print(a)
print(b)
for i in range(2, n):
    c = a + b
    print(c)
    a = b
    b = c
```

SAMPLE OUTPUT

Terminal Output

```
enter your number of terms:  7
0
1
1
2
3
5
8
```

RESULT

The program to generate the Fibonacci series was implemented and output verified successfully.

EXPERIMENT NO: 6 **Date:** _____

VOWELS AND CONSONANTS

AIM

Write a program to count number of vowels and consonants in a string

OBJECTIVES

- To iterate over character elements of a text string.
- To perform string inspection using logical OR conditions and isalpha().
- To count distinct character classes in textual input.

ALGORITHM

Step 1 :- Begin the execution of string character parsing program to count total vowels and consonants.

Step 2 :- Prompt user to enter a string and store string value in variable s.

Step 3 :- Find total character count of string s using len function and store in l.

Step 4 :- Set vowel count variable v to zero and consonant count variable c to zero.

Step 5 :- Loop through character index range from index zero up to string length limit l.

Step 6 :- Check if character at index i matches any lowercase or uppercase vowel letter.

Step 7 :- Add one to vowel counter v if character matches vowel comparison condition check.

Step 8 :- Check if character is alphabetic letter using isalpha and increment consonant counter c.

Step 9 :- Print total count of vowels and consonants to standard console terminal output window.

Step 10 :- Conclude execution of string parsing script and release string buffer memory back to system.

PROGRAM

```
s = str(input("enter a string"))
l = len(s)
c = 0
v = 0
for i in range(l):
    if s[i] == 'a' or s[i] == 'e' or s[i] == 'i' or s[i] == 'o' or s[i] == 'u' or \
        s[i] == 'A' or s[i] == 'E' or s[i] == 'I' or s[i] == 'O' or s[i] == 'U':
         v = v + 1
    elif s[i].isalpha():
         c = c + 1
print("Number of vowels is ", v)
print("Number of consonants is ", c)
```

SAMPLE OUTPUT

Terminal Output

```
enter a string:  Hello
Number of vowels is 2
Number of consonants is 3
```

RESULT

The vowel and consonant counting program was successfully executed and output verified.

EXPERIMENT NO: 7 **Date:** _____

SORTING

AIM

Write a program to Sort the elements of a list using function

OBJECTIVES

- To define modular functions accepting list parameters in Python.
- To implement bubble sort algorithm using nested loops and element swapping.
- To return and print ordered data structures.

ALGORITHM

Step 1 :- Initialize the bubble sort program to rearrange list elements in non decreasing numerical order.

Step 2 :- Declare bubble sort function taking list argument arr to perform element sorting operations.

Step 3 :- Calculate total element count of input list arr and store integer length in n.

Step 4 :- Execute outer loop for variable i ranging from index zero to length n minus one.

Step 5 :- Execute inner loop for variable j ranging from index zero to n minus i minus one.

Step 6 :- Check if element at index j is strictly greater than adjacent element at j plus one.

Step 7 :- Swap adjacent elements in place if left element is larger than right element.

Step 8 :- Return ordered list arr from bubble sort function after completing all comparison passes.

Step 9 :- Call sorting function with sample list and print sorted result to terminal window.

Step 10 :- Conclude execution of sorting algorithm program and free list memory allocation from system.

PROGRAM

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

sample_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = bubble_sort(sample_list)
print(f"Sorted list in non-decreasing order: {sorted_list}")
```

SAMPLE OUTPUT

Terminal Output

```
Sorted list in non-decreasing order:  [11, 12, 22, 25, 34, 64, 90]
```

RESULT

The list sorting function was successfully implemented using bubble sort and output verified.

EXPERIMENT NO: 8 **Date:** _____

MAXIMUM AND MINIMUM ELEMENT OF A TUPLE

AIM

Write a program to find Maximum and minimum elements of a tuple using function

OBJECTIVES

- To explore immutable sequence types (tuples) in Python.
- To traverse tuple elements and perform conditional minimum/maximum evaluation.
- To return tuple pairs containing multiple values from functions.

ALGORITHM

Step 1 :- Initialize program execution to find maximum and minimum elements of a tuple using function.

Step 2 :- Declare function find max min accepting tuple parameter tup for processing element extremes.

Step 3 :- Return None for both maximum and minimum if input tuple tup is empty.

Step 4 :- Set initial variables max element and min element to first element of tup.

Step 5 :- Iterate through each numeric element num present inside input tuple tup in order.

Step 6 :- Compare num with max element and update max element if num is strictly larger.

Step 7 :- Compare num with min element and update min element if num is strictly smaller.

Step 8 :- Return tuple containing calculated max element and min element from custom function.

Step 9 :- Pass sample tuple to function and print maximum and minimum values to terminal screen.

Step 10 :- Conclude tuple analysis program execution successfully and clear tuple structures from memory.

PROGRAM

```
def find_max_min(tup):
    if not tup:
        return None, None
    max_element = tup[0]
    min_element = tup[0]

    for num in tup:
        if num > max_element:
            max_element = num
        if num < min_element:
            min_element = num
    return max_element, min_element

```

```
sample_tuple = (3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)
max_element, min_element = find_max_min(sample_tuple)
print(f"The maximum element in the tuple is: {max_element}")
print(f"The minimum element in the tuple is: {min_element}")
```

SAMPLE OUTPUT

Terminal Output

```
The maximum element in the tuple is:  9
The minimum element in the tuple is:  1
```

RESULT

The function to find maximum and minimum elements of a tuple was executed and verified.